

UNIT 2: C PROGRAMMING

01 - Decision Making (if & if-else)

 **Aim:** To learn how programs make decisions and execute specific blocks of code based on conditions being True or False.

1. WHAT IS DECISION MAKING?

In C, decision-making statements control the execution flow. A condition is evaluated: if it is **True** (**non-zero**), the designated code executes; if **False** (**0**), that code is skipped.

1. Simple if Statement

Executes the block **only** when the condition is True.

```
int marks = 65;
if (marks ≥ 50) {
    printf("Passed!\n");
}
```

Flow:

Condition is True → Prints "Passed!"

Condition is False → Ignores the block and moves on.

2. if-else Statement

Provides two clear paths: one for True, one for False.

```
int age = 16;
if (age ≥ 18) {
    printf("Eligible to vote\n");
} else {
    printf("Not eligible\n");
}
```

Output: Not eligible

2. PRACTICAL EXAMPLE: CHECK EVEN OR ODD

```
#include <stdio.h>
int main() {
    int num = 7;
    if (num % 2 == 0) {
        printf("%d is Even\n", num);
    } else {
        printf("%d is Odd\n", num);
    }
    return 0;
}
```

3. QUICK SYNTAX GUIDE

Statement Type	When to Use?	Syntax Form
----------------	--------------	-------------

Simple if	When there is only one optional action.	<code>if (condition) { ... }</code>
if - else	When there are two mutually exclusive actions.	<code>if (...) { ... } else { ... }</code>
else-if Ladder	When there are 3 or more distinct conditions to test.	<code>if (...) { } else if (...) { } else { }</code>

⚠ Common Beginner Mistake:

Do not put a semicolon right after the `if` condition: `if (a > b);` ❌

A semicolon terminates the statement immediately, causing the block below to run unconditionally!



One-Line Revision: `if` runs code when a condition is True; `else` runs code when that condition is False.

UNIT 2: C PROGRAMMING

02 - Decision Making (switch Statement)

 **Aim:** To learn multi-way branching using **switch**, understand the role of **case**, **break**, and **default**, and avoid fall-through errors.

1. WHAT IS A SWITCH STATEMENT?

When you need to choose one action out of many fixed choices (like selecting a menu option from 1 to 7), writing multiple **else-if** statements becomes messy. A **switch statement** tests an integer or character variable against multiple constant values called **cases**.

Anatomy of switch

- **switch(exp):** Expression must evaluate to an **int** or **char**.
- **case constant:** Target value with a colon (:).
- **break;** Jumps out of the switch block once the matching case runs.
- **default:** Runs if no matching case is found (like the final **else**).

What is Fall-Through?

If you forget to write **break;** at the end of a case, C will keep running into the next cases automatically — even if their values do not match!

Always end every case with **break;** unless you intentionally want cases to share identical logic.

2. COMPLETE PRACTICAL PROGRAM – DAY OF THE WEEK

```
#include <stdio.h>
int main() {
    int day = 2;
    switch (day) {
        case 1:
            printf("Monday\n");
            break;
        case 2:
            printf("Tuesday\n");
            break;
        case 3:
            printf("Wednesday\n");
            break;
        default:
            printf("Invalid day number!\n");
    }
    return 0;
}
```

Output: Tuesday

3. SWITCH VS ELSE-IF LADDER

Feature	switch Statement	else-if Ladder
Expression Checked	Only single integer or character constant.	Any relational or logical condition (a > 5 && b < 10).
Floating Points	✗ Not allowed (no float or double).	✓ Allowed (marks ≥ 75.5).
Readability	Very clean for fixed menus / options.	Can become messy with too many branches.
Exit Control	Needs explicit break; to exit block.	Exits automatically after executing matching branch.

💡 Golden Rules of switch:

1. Case values must be **constant values or literals** (e.g., case 1: or case 'A:'), never variables like case x:.
2. Two cases cannot have duplicate values in the same switch block.



One-Line Summary: switch tests fixed integer/character values; each case needs break;, and default acts as the safety net.

UNIT 2: C PROGRAMMING

03 - Looping (while & for Loops)

 **Aim:** To master iterative control structures, explore loop lifecycles, and build strong problem-solving skills using **while** and **for** loops.

1. LOOP LIFECYCLE: THE 4 VITAL PILLARS

Every loop in C depends on **four core steps**. If any step is missing or placed incorrectly, the loop will either produce incorrect logic or get stuck forever.

1. Initialization: Sets the start count → 2. Condition: Evaluates True/False → 3. Body: Statements to repeat → 4. Update: Modifies variable (++ / --)

1. while Loop (Condition-Driven)

Ideal when repeating code until a specific state occurs (repetition count not strictly known).

```
int count = 1;           // 1.
Initialization
while (count ≤ 3) {      // 2.
  Condition
  printf("N=%d ", count); // 3.
  Loop Body
  count++;               // 4.
  Update
}
```

Output: N=1 N=2 N=3

⚡ If condition is False initially (e.g. count=5), loop body runs **0 times**.

2. for Loop (Counter-Driven)

Ideal when exact count is predetermined (e.g., looping 10 or 100 times).

```
// for(Init ; Condition ;
Update)
for (int i = 1; i ≤ 3; i++) {
  printf("N=%d ", i);
}
```

Output: N=1 N=2 N=3

⚡ Clean & compact — keeps initialization, condition, and increment together.

2. TRACE TABLE – EXECUTION STEP-BY-STEP

Students can verify exactly how the computer handles every single iteration step:

Iteration #	Variable Value	Condition Check ($i \leq 3$)	Action / Output	Update Step ($i++$)
1st Pass	$i = 1$	$1 \leq 3 \rightarrow \text{True}$	Prints N=1	i becomes 2
2nd Pass	$i = 2$	$2 \leq 3 \rightarrow \text{True}$	Prints N=2	i becomes 3
3rd Pass	$i = 3$	$3 \leq 3 \rightarrow \text{True}$	Prints N=3	i becomes 4
Loop Exit	$i = 4$	$4 \leq 3 \rightarrow \text{False}$	Terminates loop	Moves to next program statement

3. DETAILED COMPARISON

Feature	while Loop	for Loop
---------	------------	----------

Header Syntax	Contains only the test condition <code>while(cond)</code> .	Groups init, condition, and step <code>for(i; c; u)</code> .
Best Use Case	Waiting for user input, reading files, sentinel values.	Iterating through arrays, counting numbers, math tables.
Scope of Counter	Variable remains active in memory after loop exits.	If declared inside <code>for(int i ...)</code> , destroyed upon exit.
Evaluation Point	Entry-controlled (tested before entering body).	Entry-controlled (tested before entering body).

⚠ **Common Traps to Avoid:**

1. **Infinite Loop:** Forgetting `count++` means condition stays True forever!
2. **Accidental Semicolon:** `for( ... );` runs an empty loop and executes braces only once!

💡 **Practical Coding Advice:**

Choose `for` when you can calculate repetitions beforehand. Choose `while` when repetition depends entirely on conditions inside the program.



One-Line Summary: `while` and `for` test conditions before entering; if the condition is False at the start, neither loop executes even once.

UNIT 2: C PROGRAMMING

04 - Looping (do-while & Nested Loops)

 **Aim:** To explore exit-controlled loops using **do-while**, understand guaranteed single execution, and master multi-dimensional nested loops.

1. EXIT-CONTROLLED LOOPS: DO-WHILE

In a **do-while** loop, the loop statements execute **first**, and the condition is tested **afterwards** at the bottom. Because the test occurs at the end, the loop body is **guaranteed to run at least once**, even if the condition is False right from the beginning!

Guaranteed Single Run Example

Notice that `i = 10` fails `i < 5`, but still prints once:

```
int i = 10;
do {
    printf("Value: %d\n", i);
    i++;
} while (i < 5); // Notice the semicolon!
```

Output: Value: 10

Primary Use Case: Interactive Menus

Displaying options at least once before prompting the user:

```
int choice;
do {
    printf("1.Play 2.Exit → ");
    scanf("%d", &choice);
} while (choice != 2);
```

⚡ Guarantees the menu shows up at least once on screen.

2. ENTRY-CONTROLLED VS EXIT-CONTROLLED COMPARISON

Comparison Parameter	while / for Loop (Entry-Controlled)	do-while Loop (Exit-Controlled)
Condition Check Point	Tested at the beginning (before entering body).	Tested at the end (after executing body).
Minimum Executions	0 times (if condition starts as False).	1 time (guaranteed at least once).
Ending Syntax	No semicolon after while(cond).	Must end with semicolon: while(cond);
Best Suitable For	Searching, counting, iterating arrays.	Menu-driven CLI apps, re-prompting input.

3. NESTED LOOPS (LOOP INSIDE A LOOP)

A **nested loop** has an outer loop and an inner loop. For **every single turn** of the outer loop, the inner loop completes its entire cycle from start to finish.

```
// Outer loop controls Rows (i)
for (int i = 1; i ≤ 3; i++) {
    // Inner loop controls Columns (j)
```

Visualizing Rows & Columns

```
for (int j = 1; j ≤ i; j++) {  
    printf("* ");  
}  
printf("\n"); // Next line after  
inner loop  
}
```

```
*  
* *  
* * *
```

Row 1 (i=1) → Inner runs 1 time → prints *

Row 2 (i=2) → Inner runs 2 times → prints *

Row 3 (i=3) → Inner runs 3 times → prints *

⚠ Syntax Alert for do-while:

Forgetting the final semicolon while(cond); causes a compiler error! Unlike while and for, the do-while structure requires that closing semicolon.

💡 Nested Loop Mental Model:

Think of a clock: the outer loop is the **Hour Hand**, and the inner loop is the **Minute Hand**. The minute hand must complete 60 full ticks before the hour hand moves by just 1!



One-Line Summary: do-while tests at exit and runs at least once; in nested loops, the inner loop runs completely for every single outer iteration.

UNIT 2: C PROGRAMMING

05 - Jump Statements (break & continue)

 **Aim:** To control and alter loop execution paths dynamically using jump statements, specifically **break** and **continue**.

1. WHAT ARE JUMP STATEMENTS?

Normally, a loop runs predictably from start to finish. **Jump statements** let you override this flow: you can immediately stop and exit the entire loop (**break**) or bypass only the remaining statements of the current cycle and jump straight to the next pass (**continue**).

1. The break Statement (Emergency Exit)

Action: Terminates the loop immediately and jumps out to the code below the loop block.

```
for (int i = 1; i ≤ 5; i++) {  
    if (i == 3) {  
        break; // Exit loop  
        entirely  
    }  
    printf("%d ", i);  
}
```

Output: 1 2

⚡ When $i = 3$, the loop stops completely. Numbers 3, 4, 5 are never processed!

2. The continue Statement (Skip Pass)

Action: Skips the rest of the current iteration and jumps directly to the update/next test.

```
for (int i = 1; i ≤ 5; i++) {  
    if (i == 3) {  
        continue; // Skip only 3  
    }  
    printf("%d ", i);  
}
```

Output: 1 2 4 5

⚡ Only 3 is skipped! The loop continues running normally for 4 and 5.

2. COMPREHENSIVE SIDE-BY-SIDE COMPARISON

Comparison Parameter	break Statement	continue Statement
Primary Function	Terminates and destroys the loop completely.	Skips the current cycle and begins the next iteration.
Usable Contexts	Inside loops (for, while, do-while) and switch blocks.	Only valid inside looping structures (not in switch).
Subsequent Iterations	All future iterations are cancelled.	Subsequent iterations run normally.
Real-World Analogy	Pulling the emergency fire alarm to leave the building.	Skipping a damaged song track to play the next song.

3. BEHAVIOR INSIDE NESTED LOOPS

An essential rule for first-year exams: Jump statements affect **only the innermost loop** in which they reside!

```
for (int i = 1; i ≤ 2; i++) {  
    for (int j = 1; j ≤ 3; j++) {  
        if (j == 2) break; // Exits  
        inner loop only  
        printf("(%d,%d) ", i, j);  
    }  
}
```

Output & Explanation

Output: (1,1) (2,1)

Notice that **break** only stopped loop **j**. Loop **i** continued executing its second pass without interruption!

while Loop + continue Trap:

In a while loop, placing **continue** before the increment (**i++**) causes an infinite loop because **i** never updates! Always place update statements carefully.

Best Practice:

Always enclose **break** and **continue** inside an **if** condition. Writing them unconditionally makes any statements below them unreachable dead code!



One-Line Summary: **break** stops and exits the loop immediately; **continue** skips only the current pass and moves directly to the next iteration.

UNIT 2: C PROGRAMMING

06A - Function Basics (Declaration, Definition & Calling)

 **Aim:** To break large codebases into reusable sub-programs using functions, and master parameter passing and return types.

1. WHAT IS A FUNCTION? & ITS 3 CORE PILLARS

A **function** is an independent, self-contained block of code designed to perform a specific task. Instead of writing duplicate logic multiple times, you write it once inside a function and invoke it whenever needed.



1. Declaration (Prototype)

Tells the compiler the function's name, return type, and parameters **before** main().

```
int add(int, int); // ends with ;
```



2. Function Call

Transfers execution control from main() to the function to perform work.

```
result = add(10, 20);
```



3. Function Definition

Contains the actual body and lines of code that execute the logic.

```
int add(int a, int b) {  
    return a + b;  
}
```

2. COMPLETE PRACTICAL PROGRAM

```
#include <stdio.h>  
int calculateSum(int a, int b); //  
1. Prototype  
  
int main() {  
    int sum = calculateSum(15, 25); //  
2. Call  
    printf("Sum = %d\n", sum);  
    return 0;  
}  
  
// 3. Definition  
int calculateSum(int a, int b) {  
    return a + b;  
}
```

Output: Sum = 40

Execution Path:

1. main() starts running.
2. Hits calculateSum(15, 25).
3. CPU jumps down to definition, calculates 15 + 25 = 40.
4. return 40; hands value back to sum in main().

3. THE 4 COMBINATIONS OF PARAMETERS & RETURN TYPES

Category	Function Header Example	Description / Student Notes
No Argument, No Return	<code>void greet(void);</code>	Pure action only (e.g., printing a banner message).
With Argument, No Return	<code>void printAge(int age);</code>	Takes input data from caller; prints inside without sending a value back.
No Argument, With Return	<code>int getFixedCode(void);</code>	Generates/reads a value independently and returns it to caller.
With Argument, With Return	<code>int square(int n);</code>	Most common: Takes input data, calculates, and returns computed result.

⚠ Semicolon Alert:

Function declarations (prototypes) **must** end with a semicolon (`int add();`), but function definitions **must never** have one before their curly braces `{ }`!

💡 Actual vs Formal Parameters:

- **Actual Arguments:** The real values passed in the call: `add(15, 25)`.
- **Formal Parameters:** The receiver variables in the header: `add(int a, int b)`.



One-Line Summary: Declare first, call inside `main()`, and define the work in the body; **return** sends the computed answer back.

UNIT 2: C PROGRAMMING

06B - The 4 Function Categories

 **Aim:** To master the 4 core design styles of functions with clear code snippets, execution flow, and input/output behavior.

1. VISUAL CODE BLUEPRINT FOR ALL 4 TYPES

1. No Argument, No Return

 **Flow:** Function inputs, calculates & prints internally.

```
void add() {  
    int a = 10, b = 20;  
    printf("Sum = %d\n", a + b);  
}  
int main() {  
    add(); // main() passes  
nothing  
    return 0;  
}
```

Output: Sum = 30

2. With Argument, No Return

 **Flow:** main() supplies data; function prints.

```
void add(int a, int b) {  
    printf("Sum = %d\n", a + b);  
}  
int main() {  
    add(10, 20); // Passes values  
    return 0;  
}
```

Output: Sum = 30

3. No Argument, With Return

 **Flow:** Function computes internally & sends answer back.

```
int add() {  
    int a = 10, b = 20;  
    return a + b; // Sends 30  
}
```

```

back
}
int main() {
    int res = add();
    printf("Sum = %d\n", res);
    return 0;
}

```

Output: Sum = 30

4. With Argument, With Return

 **Flow:** main() gives inputs; function returns answer.

```

int add(int a, int b) {
    return a + b; // Computes &
returns
}
int main() {
    int res = add(10, 20);
    printf("Sum = %d\n", res);
    return 0;
}

```

Output: Sum = 30

2. COMPREHENSIVE COMPARISON TABLE

Category	Data Flow Direction	Return Type	Calling Syntax	Real-World Analogy
No Arg, No Return	Self-contained inside function	void	<code>greet();</code>	Ringling an alarm bell
With Arg, No Return	<code>main()</code> → Function	void	<code>displayAge(18);</code>	Putting coins into a vending display
No Arg, With Return	Function → <code>main()</code>	int, float, etc.	<code>code = getCode();</code>	Rolling a dice and noting the result
With Arg, With Return	Two-way (<code>main()</code> ↔ Function)	int, float, etc.	<code>ans = square(5);</code>	Giving raw fruit to a juicer to get juice

Simple Student Rules:

- **Argument:** Information sent *INTO* the function via parentheses (...).
- **Return Value:** The single answer sent *BACK* to the caller using **return**.
- If nothing is returned, the return type is always written as **void**.

Key Exam Insights & Advanced Design Rules:

- **Single Return Constraint:** A C function can return at most **one** value directly using **return**. To return multiple values, use pointers (Call by Reference) or structures.
- **Stack Memory Lifecycle:** Local variables inside functions exist on the stack frame and are destroyed immediately when **return** executes or the closing brace **}** is hit.
- **Parameter Matching:** Actual arguments passed in `main()` and formal parameters in the function definition must match in **number, order, and data type**.
- **Reusability & Testing:** Category 4 (*With Arg, With Return*) is preferred in software design because it decouples calculations from printing, allowing unit tests and reusability.



One-Line Summary: Arguments supply input data to functions; return types specify what data the function delivers back to the caller.

UNIT 2: C PROGRAMMING

07A - Parameter Passing (Call by Value)

 **Aim:** To understand how data is passed to functions in C using Call by Value, and why modifications to formal parameters do not affect the original variables in `main()`.

1. WHAT IS CALL BY VALUE?

In C, functions pass arguments by **value** by default. This means the compiler makes a **duplicate photocopy** of the actual argument and places it into the formal parameter's separate memory box. Any modifications made inside the called function change only that copy, leaving the original variable completely untouched!

2. CLASSIC EXPERIMENT: ATTEMPTING TO SWAP NUMBERS (FAILS IN CALL BY VALUE)

```
#include <stdio.h>

// Passes copies of values (Call by Value)
void swap(int a, int b) {
    int temp = a; // Swaps local copies only
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    printf("Before swap: x=%d, y=%d\n", x, y);
    swap(x, y); // Passes copies of 10 and 20
    printf("After swap: x=%d, y=%d\n", x, y);
    return 0;
}
```

Output:

Before swap: x=10, y=20

After swap: x=10, y=20 (Not Swapped!)

Why did x & y remain 10 & 20?

x and y live inside `main()`'s memory stack frame. Parameters a and b receive only separate duplicate copies. Swapping a and b inside `swap()` alters only the copies, leaving x and y in `main()` completely untouched!

3. VISUAL MEMORY MODEL: THE PHOTOCOPY ANALOGY

 Actual Arguments (in main)

Memory: x=#2001 (10), y=#2002 (20)

Original variables in main — untouched!

→ Copies Sent →

 Formal Parameters (in swap)

Memory: a=#6004 (10), b=#6005 (20)
Swapped inside function, then destroyed on exit.

4. ACTUAL ARGUMENTS VS FORMAL PARAMETERS

Comparison Parameter	Actual Arguments	Formal Parameters
Where are they defined?	Inside the function call statement in <code>main()</code> .	In the function declaration / header line.
Format	Can be variables, expressions, or constants: <code>swap(x, y)</code> or <code>swap(10, 20)</code> .	Must be explicit variable declarations: <code>void swap(int a, int b)</code> .
Memory Allocation	Allocated when <code>main()</code> begins execution.	Created when the function is called; destroyed immediately upon <code>return</code> .

⚠ The Swapping Trap:

Writing a simple swap function like `swap(int a, int b)` using Call by Value **will not swap** the original variables in `main()`! (Swapping originals requires pointers / Call by Reference).

💡 Why use Call by Value?

Safety & Independence: It ensures functions cannot accidentally corrupt or overwrite important data inside caller functions like `main()`.



One-Line Summary: Call by Value passes only a duplicate copy of the data; changes inside the called function never affect original variables in `main()`.

UNIT 2: C PROGRAMMING

07B - Parameter Passing (Call by Reference) (USING POINTERS)

 **Aim:** To understand how to pass memory addresses (references) to functions using pointers, enabling the function to directly modify the original variables in `main()`.

1. WHAT IS CALL BY REFERENCE?

In **Call by Reference**, instead of sending a copy of the value, we pass the **memory address (&x)** of the variable. The function receives this address inside a **pointer variable (int *ptr)**. Any change made through the dereference operator (`*ptr`) directly updates the original variable in `main()`!

2. COMPLETE PRACTICAL PROGRAM – SWAPPING TWO NUMBERS

```
#include <stdio.h>

// Takes addresses of integers as
// pointer parameters
void swap(int *a, int *b) {
    int temp = *a; // temp gets value
    // at address a
    *a = *b;        // place value of b
    // into address a
    *b = temp;      // place temp into
    // address b
}

int main() {
    int x = 10, y = 20;
    printf("Before swap: x=%d,
    y=%d\n", x, y);
    swap(&x, &y); // Pass memory
    // addresses using &
    printf("After swap: x=%d,
    y=%d\n", x, y);
    return 0;
}
```

Output:

Before swap: x=10, y=20
After swap: x=20, y=10 (Swapped!)

Why did x & y change?

`swap(&x, &y)` gave the function the keys to x and y's original memory cells. Writing `*a = *b` modified the value directly inside memory slot #2001!

3. VISUAL MEMORY MODEL: THE DIRECT KEY ANALOGY

 **Original Variable (x in main)**

Memory Address: **#2001**

Value: **10** → **updated to 20**

The original storage box in RAM.

🔑 Pointer Parameter (*a in swap)

Stored Value: **#2001** (Address of x)

Operation: ***a = 20;**

Accesses box #2001 directly via pointer!

4. CALL BY VALUE VS CALL BY REFERENCE

Comparison Parameter	Call by Value	Call by Reference
What is passed?	Copy of the value (e.g., swap(x, y)).	Memory address of the variable (e.g., swap(&x, &y)).
Parameter Declaration	Normal variable: void swap(int a, int b).	Pointer variable: void swap(int *a, int *b).
Original Variable Effect	Original variable in main() is not affected .	Original variable in main() is directly updated .
Primary Advantage	Safe; protects source data from being overwritten.	Can modify multiple variables and return multiple values.

⚠ Common Pointer Traps:

1. Forgetting & in function call: `swap(x, y);` passes values, causing type mismatch!
2. Forgetting * in function body: writing `a = b;` swaps memory addresses, not the actual values!

💡 Two Key Operators:

- **& (Address-Of):** Extracts the memory address of a variable.
- *** (Dereference / Value-At):** Accesses or modifies the value at that stored memory address.

📄 **One-Line Summary:** Call by Reference passes memory addresses using & and receives them with pointers *, allowing functions to directly modify the original variables.

UNIT 2: C PROGRAMMING

08 - Recursion

 **Aim:** To understand recursion (a function calling itself), identify the critical necessity of a base condition, and trace execution stacks step-by-step.

1. WHAT IS RECURSION? & ITS 2 ESSENTIAL PARTS

Recursion is a programming technique where a function solves a problem by **calling a smaller version of itself**. Every recursive function must contain two parts:

1. The Base Condition (Stopping Point)

The condition that terminates further recursive calls and returns a direct, simple answer without calling the function again.

```
if (n == 1 || n == 0) return
1;
```

2. The Recursive Step (Self Call)

The statement where the function calls itself with a reduced parameter, progressing closer to the base condition.

```
return n * factorial(n - 1);
```

2. PRACTICAL EXAMPLE: FACTORIAL OF A NUMBER

```
#include <stdio.h>

int factorial(int n) {
    if (n <= 1) {                // 1. Base
Condition
        return 1;
    } else {
        return n * factorial(n - 1); //
2. Recursive Call
    }
}

int main() {
    int n = 4;
    printf("Factorial of %d = %d\n",
n, factorial(n));
    return 0;
}
```

 **Output:**
Factorial of 4 = 24

Math Principle:

$4! = 4 \times 3!$
 $3! = 3 \times 2!$
 $2! = 2 \times 1!$
 $1! = 1$ (Base condition reached!)

3. VISUAL STACK FLOW: WINDING VS UNWINDING

Recursion works in two phases: **Winding** (calling down to the base case) and **Unwinding** (multiplying back up):

▼ Step 1 (Call)
factorial(4)
waits for $4 * \text{fact}(3)$

▼ Step 2 (Call)
factorial(3)
waits for $3 * \text{fact}(2)$

▼ Step 3 (Call)
factorial(2)
waits for $2 * \text{fact}(1)$

🎯 Base Hit
factorial(1)
returns 1

▲ Unwind (Return)
 $2 * 1 = 2$
 $3 * 2 = 6$
 $4 * 6 = 24$

4. RECURSION VS ITERATION (LOOPS)

Comparison Parameter	Recursion	Iteration (Loops)
Execution Style	Function repeatedly calls itself.	A set of statements repeats using a loop counter.
Stopping Mechanism	Terminates when Base Condition becomes true.	Terminates when Loop Condition becomes false.
Memory Overhead	High (each call creates a new stack frame in memory).	Low (reuses same memory addresses for loop variables).
Code Cleanliness	Compact, elegant mathematical representations.	Longer code structure with counter updates.

⚠️ **Stack Overflow Danger:**
If you omit the base condition, the function calls itself infinitely until the computer runs out of stack memory, resulting in a program crash (**Segmentation Fault / Stack Overflow**)!

💡 **Exam Memory Trick:**
Always write the base condition on the **very first line** inside your recursive function. Treat it like the emergency brake on a train!

📄 **One-Line Summary:** Recursion is a function calling itself; it winds down until it hits the base condition, then unwinds back up with the final answer.

UNIT 2: C PROGRAMMING

09 - Scope and Lifetime of Variables

 **Aim:** To understand variable visibility (Scope), how long variables survive in memory (Lifetime), and how storage classes define their behavior.

1. SCOPE VS. LIFETIME – FUNDAMENTAL CONCEPTS

 **Scope** (Visibility / Where?)

The specific region of code where a variable can be seen, read, or modified (e.g., inside one function vs. the entire file).

 **Lifetime** (Duration / How Long?)

The period of time during program execution that a variable stays allocated in RAM before being destroyed.

2. LOCAL VS. GLOBAL VARIABLES – CODE EXAMPLE

```
#include <stdio.h>

int global_var = 100; // Global
Scope (accessible anywhere)

void demoFunction() {
    int local_var = 10; // Local to
demoFunction only
    printf("Inside: local=%d,
global=%d\n", local_var,
global_var);
}

int main() {
    demoFunction();
    // printf("%d", local_var); //
ERROR! local_var is dead here
    printf("Main: global=%d\n",
global_var);
    return 0;
}
```

Output:

Inside: local=10, global=100

Main: global=100

What Happened?

- `local_var` was born when `demoFunction()` was called and destroyed when it returned.
- `global_var` remained alive across the whole program.

3. SUMMARY TABLE: LOCAL VS. GLOBAL VARIABLES

Feature	Local Variable	Global Variable
Where Declared?	Inside a function or block { }.	Outside all functions (top of the file).
Scope (Visibility)	Visible only inside its enclosing { }.	Visible throughout all functions in the program.

Lifetime (Memory)	Created on function entry, destroyed on return.	Stays in memory from program start to end.
Default Initial Value	Garbage value (random junk numbers).	Zero (0) automatically initialized.

4. THE 4 STORAGE CLASSES IN C

Keyword	Storage Location	Default Value	Scope	Lifetime
auto	Stack RAM	Garbage	Local block	Function execution only (default for locals)
static	Data Segment	Zero (0)	Local block	Entire program duration (retains value!)
register	CPU Register	Garbage	Local block	Fastest access (no & address operator)
extern	Data Segment	Zero (0)	Global	Shares global variables across multiple files

★ **The static Superpower:**
A `static int count = 0;` inside a function does **not reset** when the function finishes. It retains its modified value for subsequent function calls!

⚠ **Exam Warning:**
Avoid unnecessary global variables! Any function can alter them, making debugging large programs difficult. Always prefer local variables when possible.

 **One-Line Summary:** Scope determines where a variable is accessible; Lifetime determines how long it exists in RAM.

UNIT 2: C PROGRAMMING

10 - Header Files

 **Aim:** To understand header files (.h), explore how preprocessor directives (`#include`, `#define`) function before compilation, and examine standard C libraries.

1. WHAT IS A HEADER FILE? & PREPROCESSOR DIRECTIVES

A **header file** (ending in `.h`) contains function declarations, macros, and definitions shared across programs. Lines starting with `#` are handled by the **C Preprocessor** — a tool that scans and replaces text in your source file *before* the actual compiler generates machine code.



Angle Brackets vs Quotes

• `#include <stdio.h>`^[cite: 1, 3]

Angle brackets tell the preprocessor to look inside the **system compiler directory** for standard libraries^[cite: 1, 3].

• `#include "myfunctions.h"`

Double quotes tell it to search the **current working directory** first for custom, user-created header files.

⚙️ Macro Definition (`#define`)

Used to create constant values or short shorthand functions without using memory:

```
#define PI 3.14159
#define SQUARE(x) ((x) * (x))
```

⚡ The preprocessor simply substitutes text before compilation begins.

2. PRACTICAL CODE EXAMPLE – STANDARD MATH & MACROS

```
#include <stdio.h> // For printf &
scanf[cite: 1, 3]
#include <math.h> // For sqrt() &
pow()

#define MAX_LIMIT 100

int main() {
    double value = 16.0;
    double root = sqrt(value); // From
    math.h

    printf("Square root of %.1f =
    %.2f\n", value, root);
    printf("Max limit defined is:
    %d\n", MAX_LIMIT);
    return 0;
}
```

Output:

Square root of 16.0 = 4.00
Max limit defined is: 100

Compilation Flow:

1. Preprocessor pastes contents of `stdio.h` & `math.h` directly into your code^[cite: 1, 3].
2. Replaces all instances of `MAX_LIMIT` with `100`.
3. Hands pure C code to the compiler to generate machine instructions.

3. MOST COMMONLY USED STANDARD C HEADER FILES

Header File	Full Name	Key Functions Included
<code><stdio.h></code> ^[cite: 1, 3]	Standard Input Output	<code>printf()</code> , <code>scanf()</code> , <code>getchar()</code> , <code>fopen()</code> ^[cite: 1]
<code><stdlib.h></code>	Standard General Utilities	<code>malloc()</code> , <code>free()</code> , <code>exit()</code> , <code>rand()</code> , <code>abs()</code>
<code><string.h></code>	String Manipulation	<code>strlen()</code> , <code>strcpy()</code> , <code>strcat()</code> , <code>strcmp()</code>
<code><math.h></code>	Mathematical Operations	<code>sqrt()</code> , <code>pow()</code> , <code>ceil()</code> , <code>floor()</code> , <code>sin()</code>
<code><ctype.h></code>	Character Type Classification	<code>toupper()</code> , <code>tolower()</code> , <code>isdigit()</code> , <code>isalpha()</code>

⚠ Never Put a Semicolon on Directives:

Writing `#include <stdio.h>;` ❌ or `#define PI 3.14;` ❌ produces severe syntax errors! Preprocessor directives are text-replacement instructions, not standard C statements.

💡 Header Guard Pattern (Bonus Exam Tip):

To stop duplicate inclusions of custom headers, developers wrap them in guards:
`#ifndef MYFILE_H → #define MYFILE_H →`
`#endif`

 **One-Line Summary:** Header files contain pre-built declarations; preprocessor directives (#) copy definitions and expand macros before compilation starts^[cite: 1, 3].